UET.CS2043 - Lập trình nâng cao

Lập trình mạng

Nội dung

- Tổng quan về lập trình mạng
- Mô hình Client – Server
- Quy trình kết nối TCP
- Lập trình Socket
- Giới thiệu về Rest API
- Đa luồng trong lập trình mạng

Tại sao cần lập trình mạng?

- Phần lớn các ứng dụng hiện đại là ứng dụng mạng: các trang web, các ứng dụng mobile, game online, v.v.
- Giao tiếp giữa các dịch vụ trong các hệ thống lớn
- Kết nối các thiết bị IoT với nhau
- Và nhiều ứng dụng khác

Bài toán

- Cần xây dựng một ứng dụng chat nội bộ
- Bắt đầu từ con số 0 thì làm như thế nào?

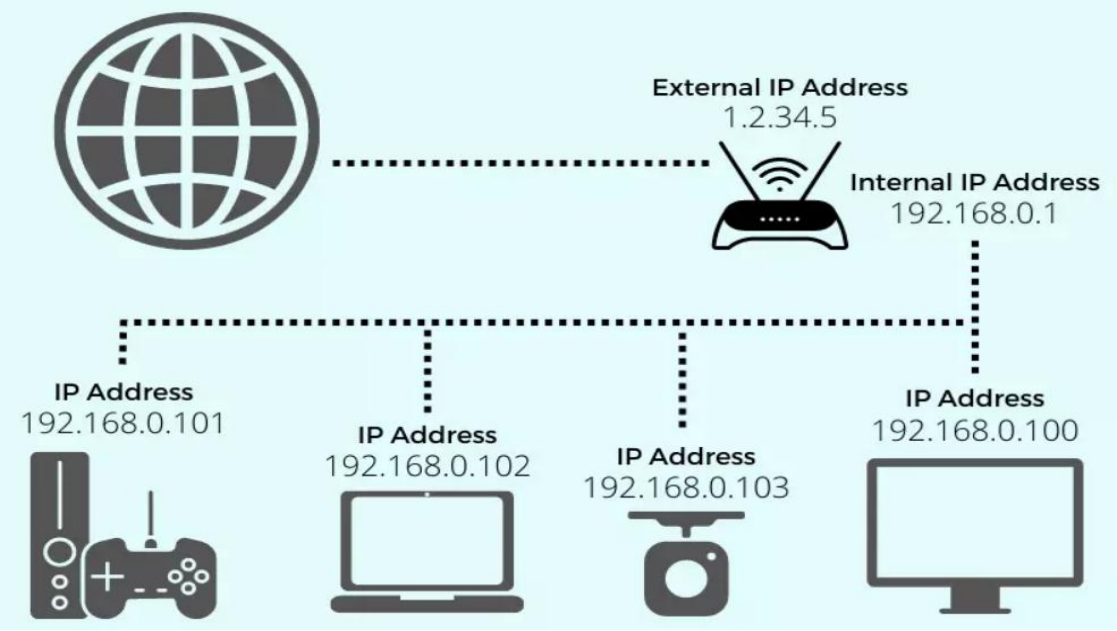
Vấn đề đầu tiên

- Máy tính của người dùng (Client) muốn nói chuyện với máy chủ (Server). Nhưng làm sao Client biết Server ở đâu?

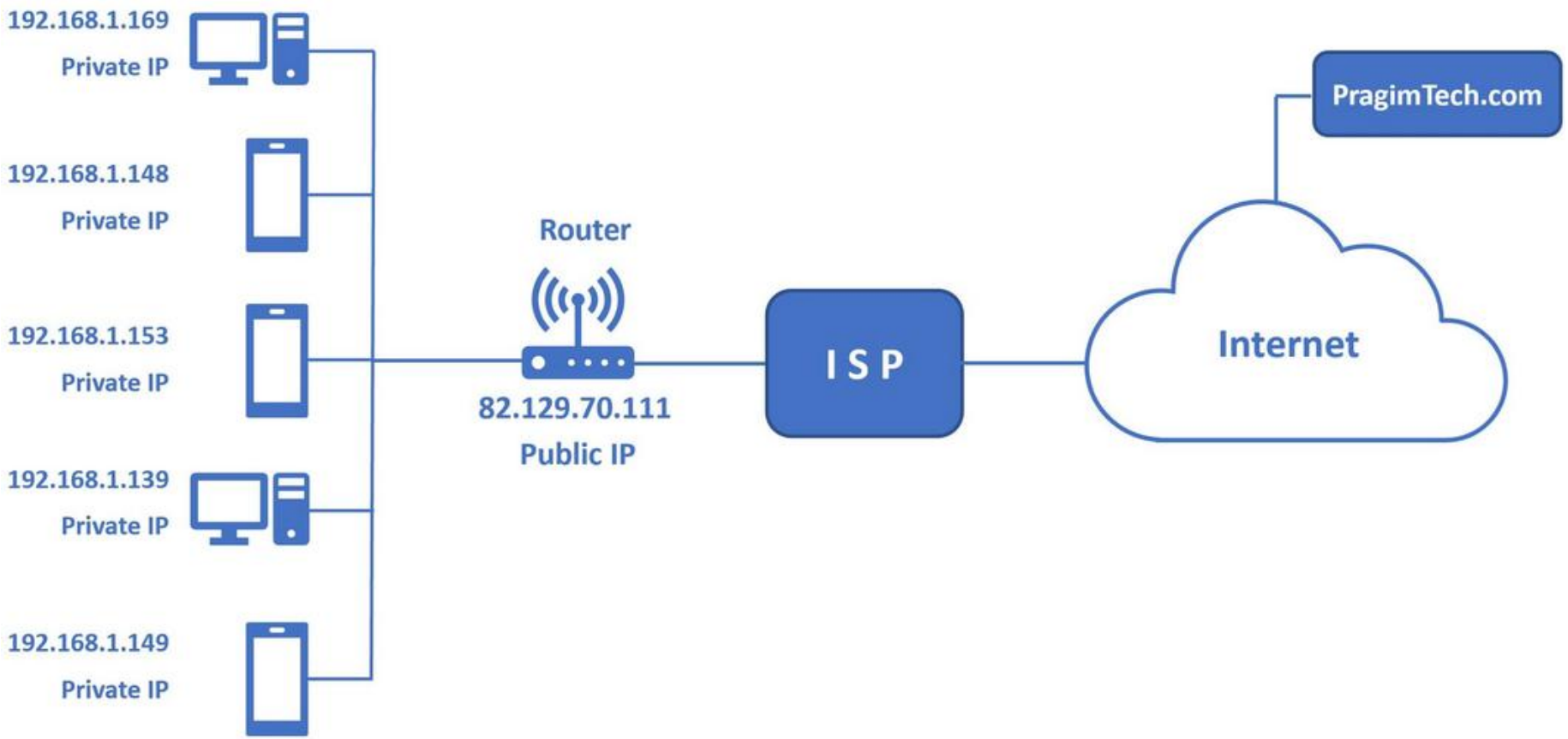
=> Cần biết được địa chỉ.

Địa chỉ IP

- IP = Internet Protocol
- IP giống như địa chỉ nhà
- Có hai loại địa chỉ: public và private
- Có hai định dạng địa chỉ: IPv4 (32-bit) và IPv6 (128-bit)



Public IP vs Private IP



Public IP vs Private IP

Public IP	Private IP
Được cấp bởi nhà mạng (Internet Service Provider – ISP)	Được cấp cho các thiết bị trong một mạng nội bộ bởi router (bộ định tuyến) hoặc một thiết bị quản lý khác
Là địa chỉ được định danh trên toàn mạng internet, có hiệu lực trên toàn bộ mạng	Chỉ có hiệu lực trong mạng nội bộ, không thể dùng được trên mạng internet

Định dạng IPv4

104 . 25 . 221 . 55

01101000.00011001.11011101.00110111

32-bit (4 bytes)

Định dạng IPv6

2342:0415:2CA1:0000:0000:0566:5671:23b4

16-bit 16-bit 16-bit 16-bit 16-bit 16-bit 16-bit 16-bit

128-bit (16 bytes)

Port (số hiệu cổng)

- Dùng để định danh người nhận, phân biệt các ứng dụng trên cùng một IP
- Ví dụ: Trình duyệt dùng port 80/443, Database dùng 3306
- Phạm vi: 0 - 65535

Giao thức TCP

- TCP = Transmission Control Protocol
- Thuộc loại giao tiếp hướng kết nối
- Đảm bảo dữ liệu đến đúng nơi, đúng thứ tự, không mất mát
- Tốc độ không quá nhanh

Giao thức UDP

- UDP = User Datagram Protocol
- Thuộc loại giao tiếp không kết nối
- Nhanh nhưng không đảm bảo độ tin cậy
- Ứng dụng: video streaming, game online, voice calls, v.v.

Giao thức nào cho ứng dụng chat?

Mô hình mạng TCP/IP

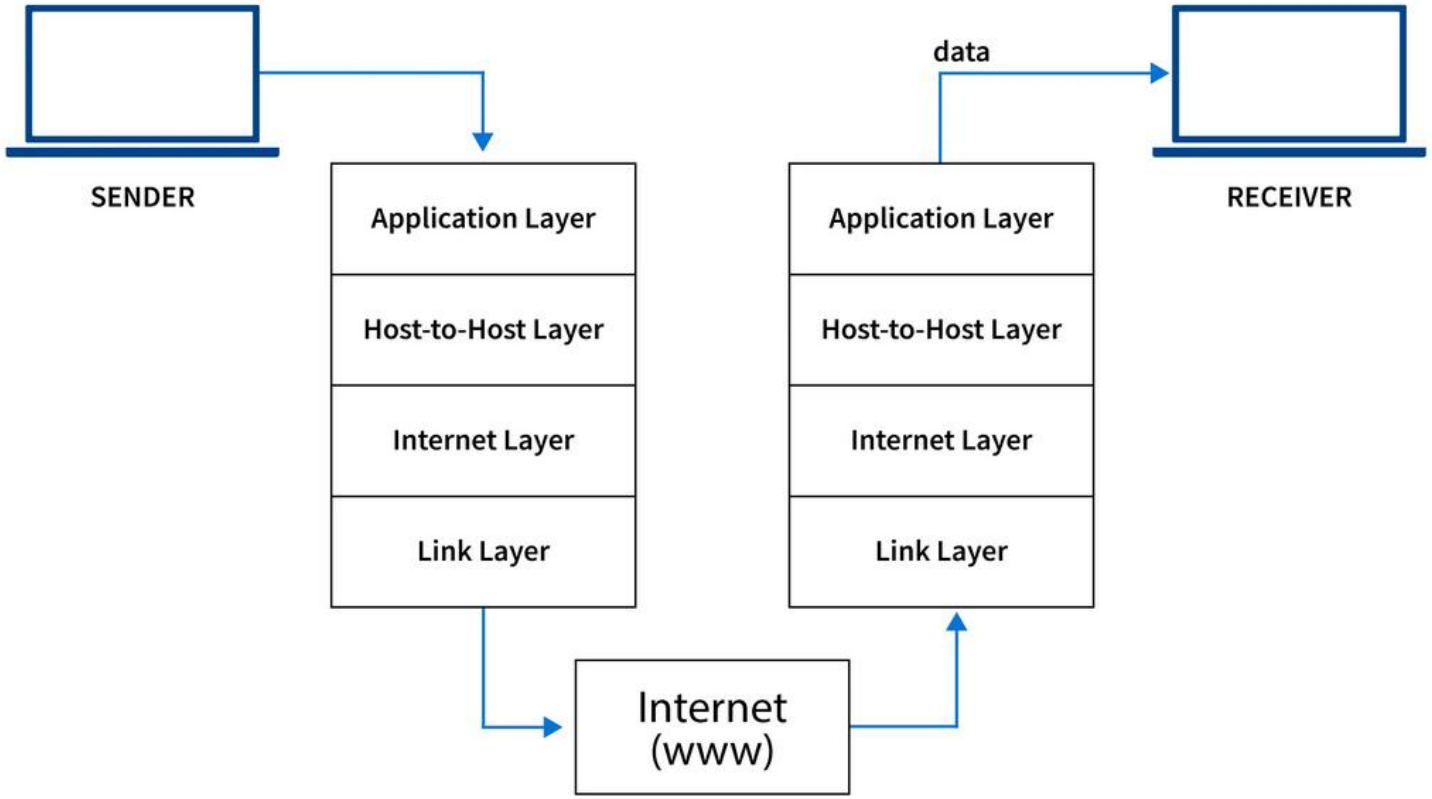
Application layer

Transport layer

Internet layer

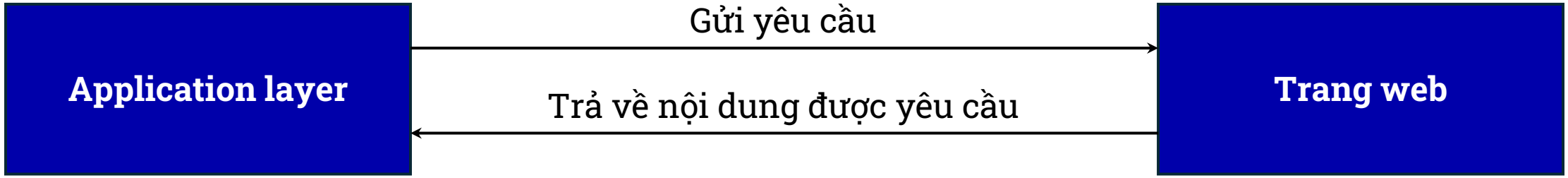
Network access
(Link layer)

Mô hình mạng TCP/IP



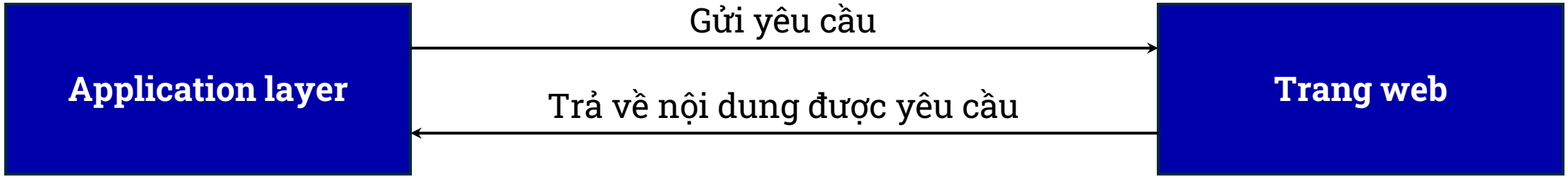
Application layer

- Gần với người dùng nhất
- Đây là nơi mà các ứng dụng như trình duyệt hay ứng dụng mobile kết nối với mạng internet



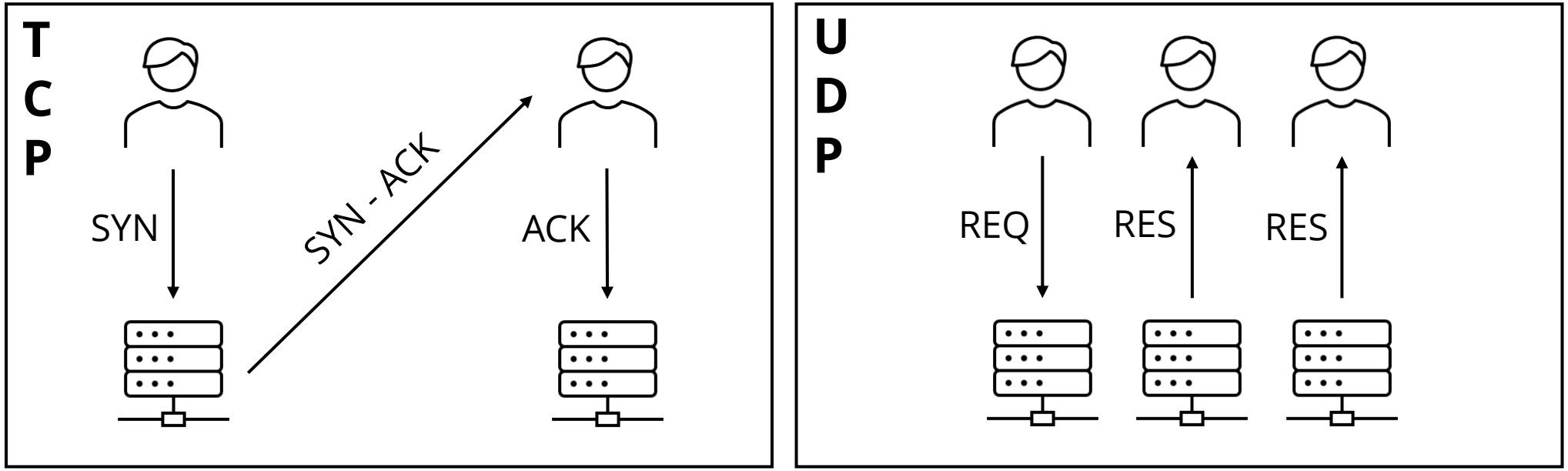
Application layer

- Gần với người dùng nhất
- Đây là nơi mà các ứng dụng như trình duyệt hay ứng dụng mobile kết nối với mạng internet



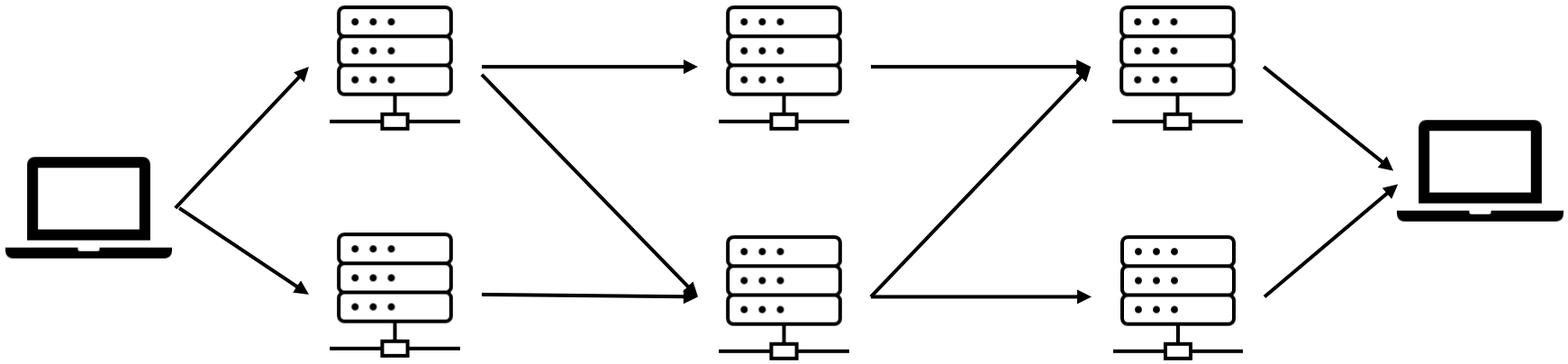
Transport layer

- Có nhiệm vụ đảm bảo dữ liệu được gửi đi đúng cách (thường sử dụng TCP hoặc UDP)



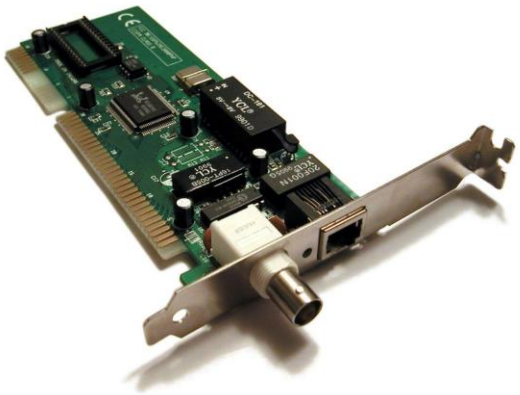
Internet layer

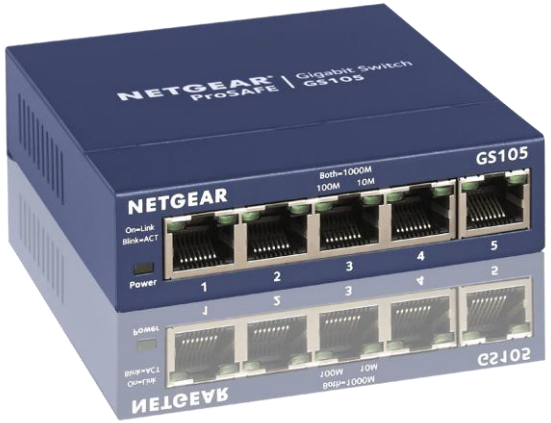
- Có nhiệm vụ quản lý đường đi của dữ liệu
- Sử dụng địa chỉ để xác định các thiết bị trên mạng
- Tính toán đường đi tốt nhất để gửi các gói tin đến đích



Network access (Link layer)

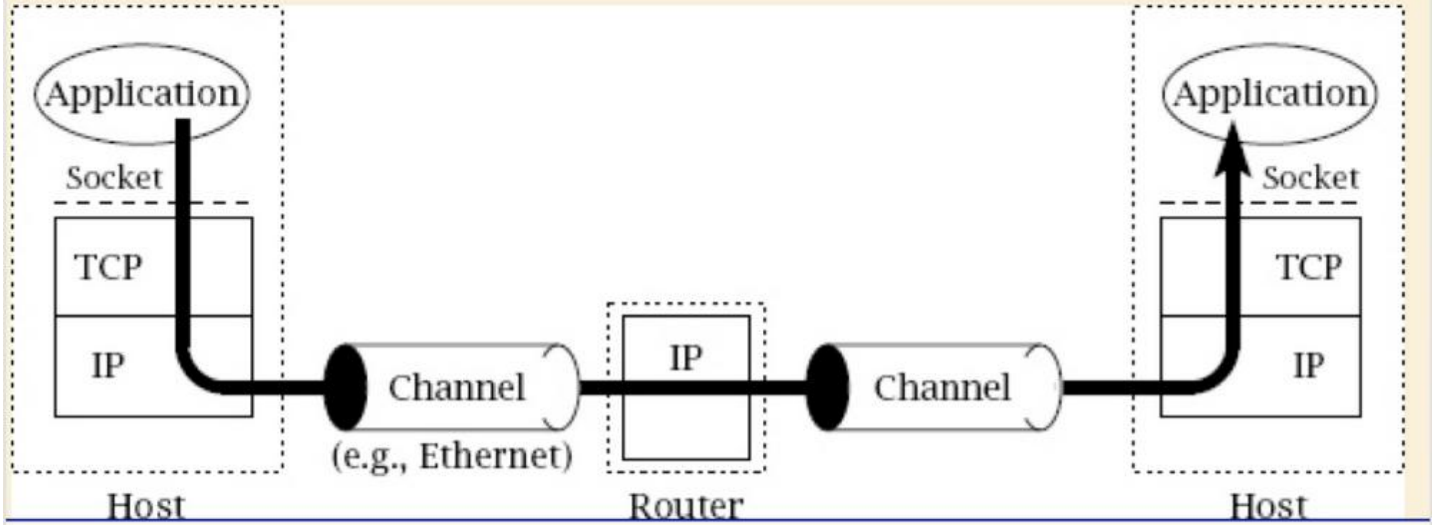
- Quản lý các tín hiệu vật lý được gửi đi và nhận về
- Bao gồm các bộ xử lý được nhúng trong các thiết bị mạng





Socket

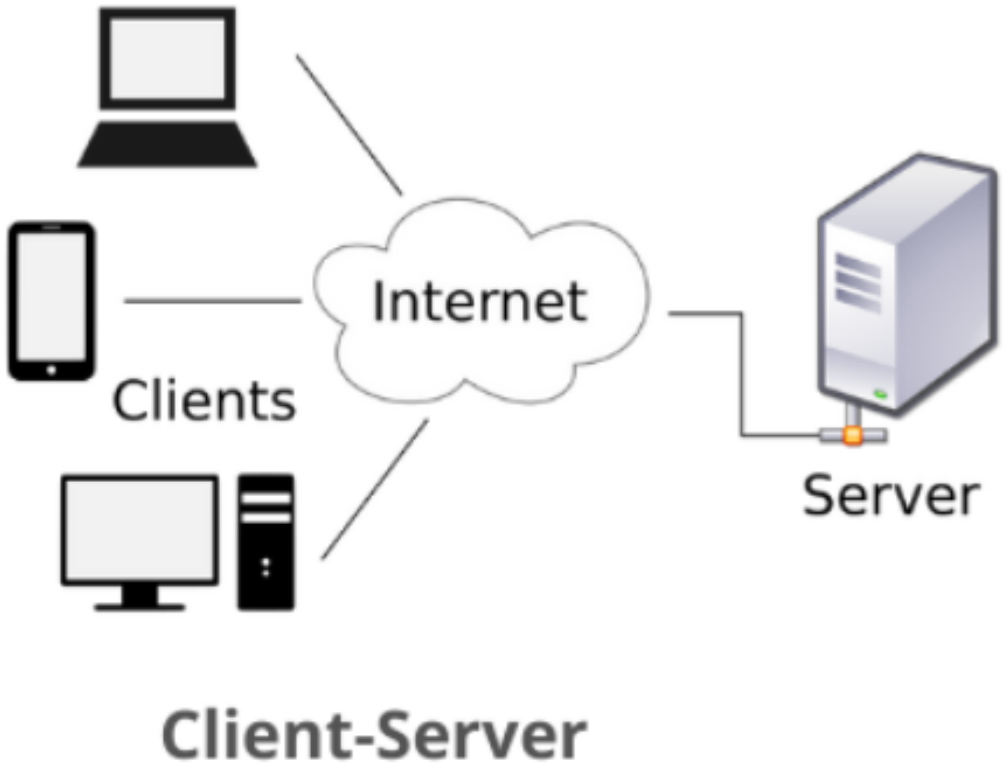
- Socket = Địa chỉ IP + Port
- Đóng vai trò là "ổ cắm" để các ứng dụng kết nối vào đường ống mạng



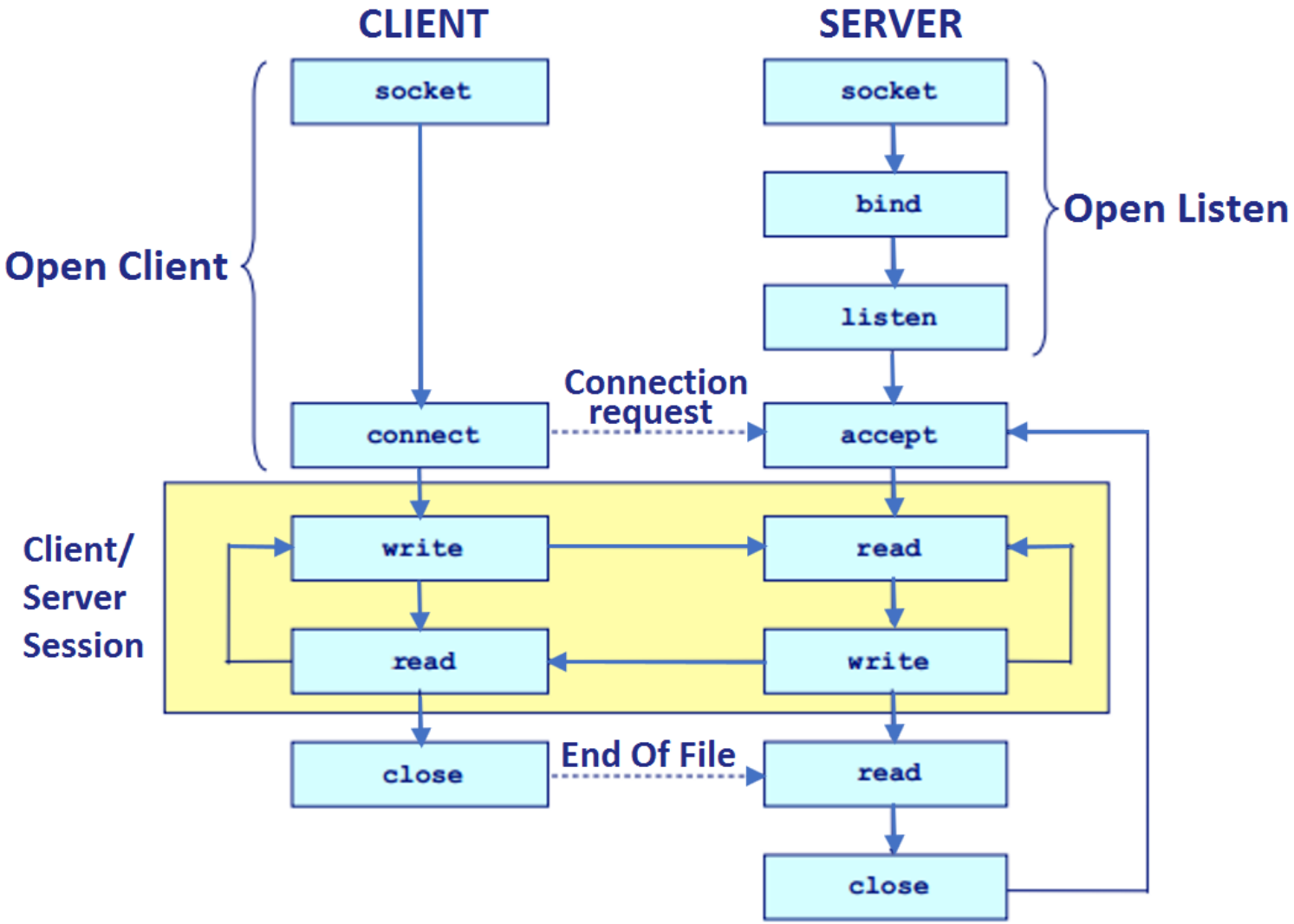
Adapted from: Donahoo, Michael J., and Kenneth L. Calvert. TCP/IP sockets in C: practical guide for programmers. Morgan Kaufmann, 2009.

Mô hình Client – Server

- Server: Cung cấp tài nguyên/dịch vụ, luôn ở trạng thái chờ
- Client: Sử dụng tài nguyên, chủ động yêu cầu



Quy trình hoạt động của mô hìnhClient – Server



Quy trình hoạt động của Server

- Binding to Port (Khởi tạo)
- Listening (Lắng nghe)
- Accepting (Chấp nhận kết nối)
- Serving (Xử lý yêu cầu)

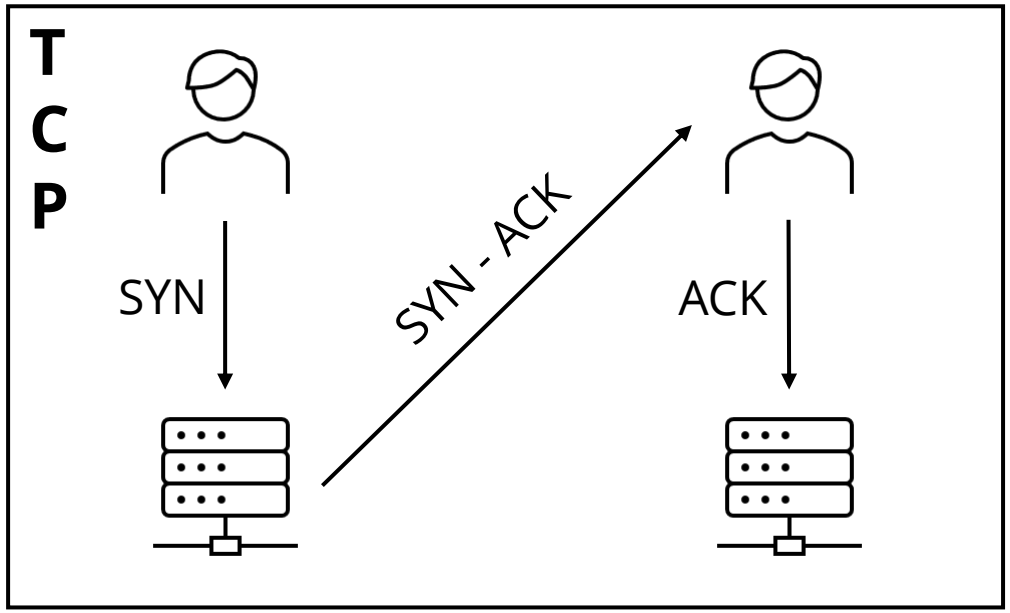
Quy trình hoạt động của Client

- Xác định IP và Port của Server
- Connect (Yêu cầu kết nối)
- Read/Write (Trao đổi dữ liệu)
- Close (Đóng kết nối)

Cơ chế Handshake của giao thức TCP

1. Client gửi SYN
2. Server gửi SYN-ACK
3. Client gửi ACK

- Mục đích là thiết lập sự đồng bộ và tin cậy trước khi truyền dữ liệu



Stream (luồng dữ liệu)

- Input Stream: Dữ liệu đi vào ứng dụng từ mạng
- Output Stream: Dữ liệu đi ra khỏi ứng dụng vào mạng

Ứng dụng Echo

- Client gửi “Hello”
- Server trả lại “Hello”

Khởi tạo Server

```
// B1. Mở cổng 5000 để lắng nghe
// ServerSocket giống như việc mở cửa tiệm
ServerSocket serverSocket = new ServerSocket(5000);
System.out.println("Server đang khởi động tại port 5000...");

// B2. Chờ khách hàng kết nối
// Dòng này sẽ BLOCK (dừng chương trình) cho đến khi có khách
Socket clientSocket = serverSocket.accept();

System.out.println("Đã có khách kết nối: " + clientSocket);
```

Thiết lập luồng (tại Server)

```
// Lấy luồng từ Socket của Client
InputStream is = clientSocket.getInputStream();
OutputStream os = clientSocket.getOutputStream();

// Bọc luồng để xử lý text thay vì byte
// BufferedReader: Giúp đọc theo từng dòng với readLine
BufferedReader reader = new BufferedReader(new
InputStreamReader(is));

// PrintWriter: Dùng để gửi text đi
// 'true' nghĩa là auto-flush, đẩy dữ liệu đi ngay lập tức
PrintWriter writer = new PrintWriter(os, true);
```

Vòng lặp chính (tại Server)

```
String line;
// Vòng lặp: Đọc dữ liệu cho đến khi client ngắt kết nối
(line == null)
while ((line = reader.readLine()) != null) {
    System.out.println("Client nói: " + line);
    writer.println("Echo: " + line); // Gửi phản hồi vào
writer
}

clientSocket.close(); // Đóng kết nối khi xong việc
```

Khởi tạo Client

```
// Kết nối đến Server ở localhost, cổng 5000
Socket socket = new Socket("127.0.0.1", 5000);
```

Thiết lập luồng (tại Client)

```
// Luồng đọc dữ liệu từ Server
BufferedReader networkIn = new BufferedReader(
    new InputStreamReader(socket.getInputStream()));

// Luồng gửi dữ liệu lên Server
PrintWriter networkOut = new
PrintWriter(socket.getOutputStream(), true);

// Luồng đọc từ bàn phím người dùng
BufferedReader userIn = new BufferedReader(
    new InputStreamReader(System.in));
```

Vòng lặp chính (tại Client)

```
String userInput;


while ((userInput = userIn.readLine()) != null) {
    networkOut.println(userInput); // Gửi đi
    System.out.println(networkIn.readLine()); // Đọc phản
hồi
}
```

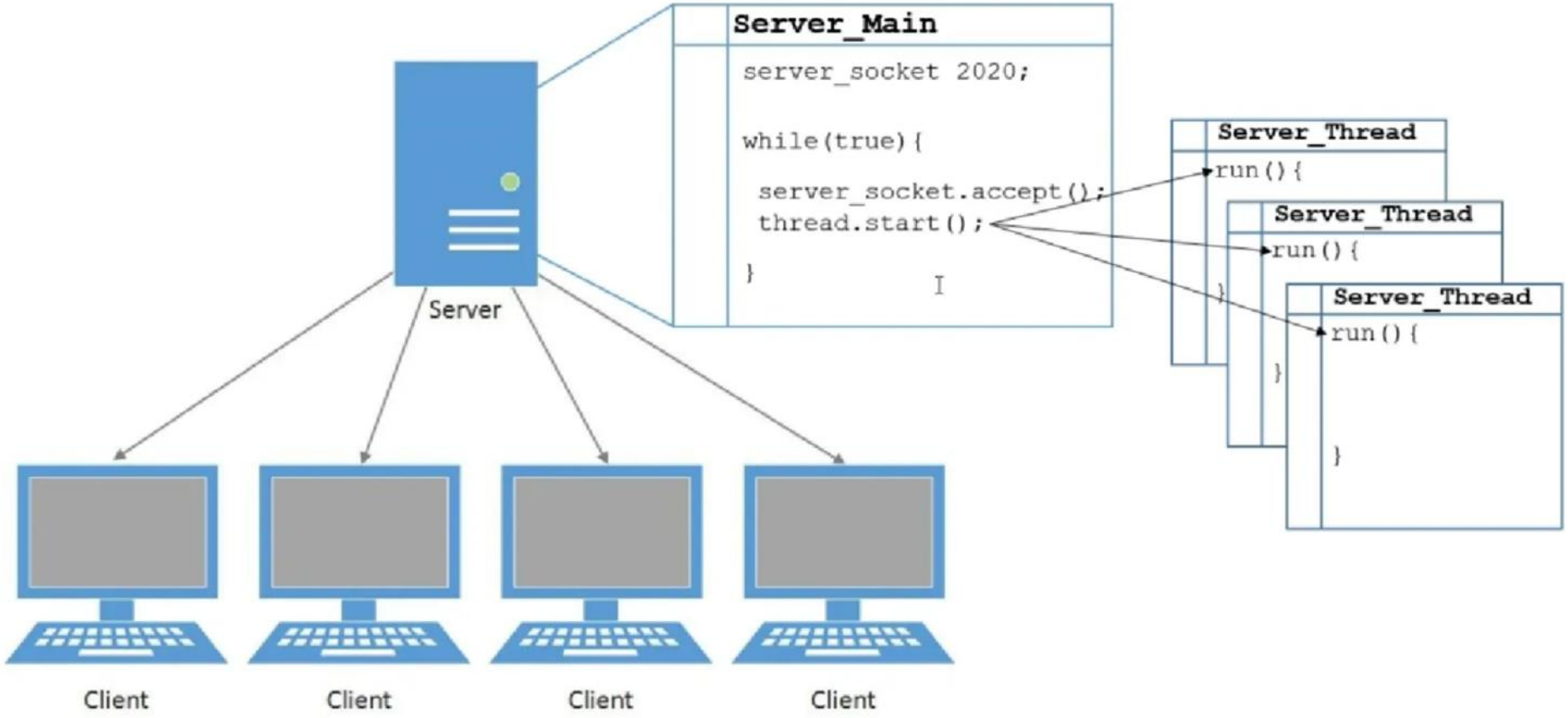
Yêu cầu mới

- Nhiều Client cùng chat với Server
- Thử:
  - Bật Server
  - Bật Client A -> Chat OK
  - Bật Client B -> Đứng hình và không nhận được phản hồi

Nguyên nhân

```
...

Socket clientSocket = serverSocket.accept();

...

while ((line = reader.readLine()) != null) {
    System.out.println("Client nói: " + line);
    writer.println("Echo: " + line);
}
```

Hướng giải quyết – Đa luồng



Hướng giải quyết – Đa luồng

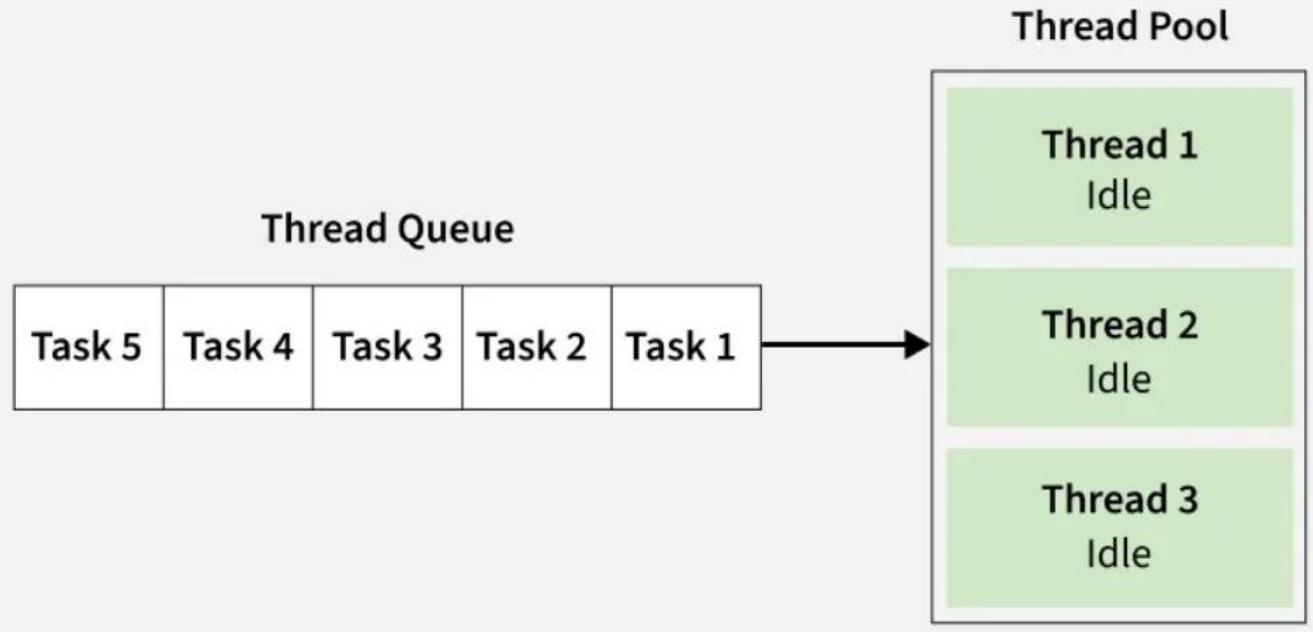
```
// Class này chịu trách nhiệm phục vụ MỘT khách hàng cụ thể
public class ClientHandler implements Runnable {
    private Socket clientSocket;
    public ClientHandler(Socket socket) {
        this.clientSocket = socket;
    }

    @Override
    public void run() { ... } // Logic đọc/ghi chuyển vào đây
}
```

Hướng giải quyết – Đa luồng

```
ServerSocket server = new ServerSocket(5000);
while (true) {
    Socket client = server.accept();
    System.out.println("Client mới: " +
client.getInetAddress());
    // Tạo một nhân viên (Thread) riêng cho khách này
    ClientHandler handler = new ClientHandler(client);
    Thread thread = new Thread(handler);
    // Bắt đầu phục vụ (start thread) để Main Thread quay lại
đón khách khác
    thread.start();
}
```

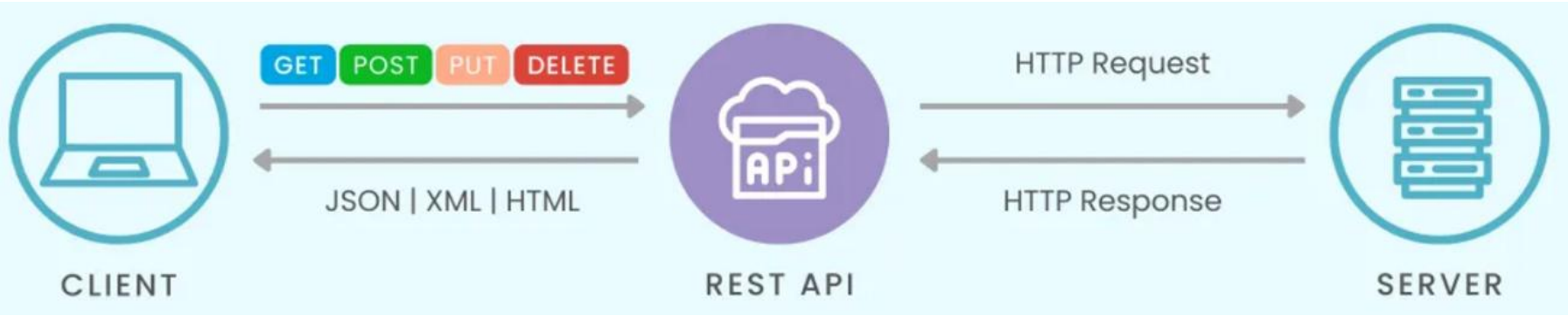
Vấn đề tiềm năng

- Nếu 1 triệu người gọi đến thì sao?
- Tạo 1 triệu Thread sẽ tràn RAM



Rest API (Representational State Transfer)

- Xây dựng dựa trên giao thức HTTP (tầng Application)
- Sử dụng các phương thức chuẩn: GET (Lấy dữ liệu), POST (Tạo mới), PUT (Cập nhật), DELETE (Xóa)
- Dữ liệu thường được đóng gói dạng **JSON** thay vì luồng byte thô

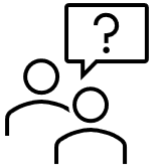


Khi nào dùng Socket? Khi nào dùng REST?

- Lập trình Socket:
  - **Ưu điểm**: Thời gian thực, độ trễ cực thấp, kết nối duy trì liên tục
  - **Ứng dụng**: Game online, ứng dụng Chat trực tuyến, hệ thống IoT truyền dữ liệu liên tục
- REST API:
  - **Ưu điểm**: Dễ triển khai, tương thích tốt với mọi nền tảng (web, mobile), không cần duy trì kết nối (stateless)
  - **Ứng dụng**: Các ứng dụng web thông thường, app mạng xã hội (đọc tin tức, đăng bài), hệ thống quản lý

Example

Tổng kết

- Tổng quan về lập trình mạng
- Mô hình Client – Server
- Quy trình kết nối TCP
- Lập trình Socket
- Giới thiệu về Rest API
- Đa luồng trong lập trình mạng

Hỏi đáp